the member functions and constructor of `CX`:

```
class CX
{
private:
    int a;
    int b;
public:
    CX() = default;
    constexpr CX(int a_, int b_):
        a(a_),b(b_)
    {}
    constexpr int get_a() const      <-- (1)
    {
        return a;
    }
    constexpr int get_b()            <-- (2)
    {
        return b;
    }
    constexpr int foo()
    {
        return a+b;
    }
};
```

Note that the `const` qualification on `get_a()` (1) is now superfluous, because it's implied by the use of `constexpr`. `get_b()` is thus `const` even though the `const` qualification is omitted (2). This now allows more complex `constexpr` functions such as the following:

```
constexpr CX make_cx(int a)
{
    return CX(a,1);
}
constexpr CX half_double(CX old)
{
    return CX(old.get_a()/2,old.get_b()*2);
}
constexpr int foo_squared(CX val)
{
    return square(val.foo());
}
int array[foo_squared(half_double(make_cx(10)))];   <-- 49 elements
```